

Introduction to PIC Programming

Programming Mid-Range PICs in C

by David Meiklejohn, Gooligum Electronics

Lesson 6: Analog Comparators

Analog comparators are used to detect whether an input signal is above or below a threshold, which may be fixed (generated by a voltage reference) or variable (perhaps another analog input).

[Mid-range lessons 9](#) and [11](#) explained how to use the type of comparator module and programmable voltage reference available on older mid-range PIC devices¹, such as the PIC12F629 and PIC16F684, using assembly language. This lesson demonstrates how to use C to access those facilities, re-implementing the examples from those assembler lessons using Microchip's XC8 compiler² (running in "Free mode"), as usual.

In summary, this lesson covers:

- Basic use of the comparator modules available on the PIC12F629 and PIC16F684
- Adding comparator hysteresis
- Comparator-driven interrupts
- Wake-up on comparator change with single and dual comparators
- Using the programmable voltage reference
- Using a single comparator to:
 - Compare a signal against a range of input thresholds
 - Compare two independent inputs against a common reference
- Using a dual comparator module with four independent inputs

Comparators

As we saw in [mid-range lesson 9](#), an *analog comparator* is a device which compares the voltages present on its positive and negative inputs.

In normal (non-inverted) operation, the comparator's output is set to a logical "high" only when the voltage on the positive input is greater than that on the negative input; otherwise the output is "low".

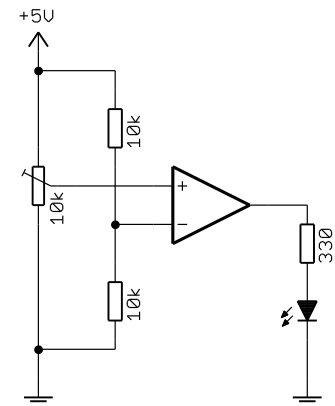
As such, they provide an interface between analog and digital circuitry.

¹ Newer and enhanced mid-range PICs, such as the 16F690 and 16F1828, include a more flexible type of comparator module, which will be covered in the enhanced mid-range tutorial series.

² Available as a free download from www.microchip.com.

In the circuit shown on the right, the comparator output will go high, lighting the LED, only when the potentiometer is set to a position past “half-way”, i.e. the positive input is greater than 2.5 V.

Comparators are typically used to detect when an analog input is above or below some threshold (or, if two comparators are used, within a defined band) – very useful for working with many types of real-world sensors. They are also used with digital inputs to match different logic levels, and to shape poorly defined signals.

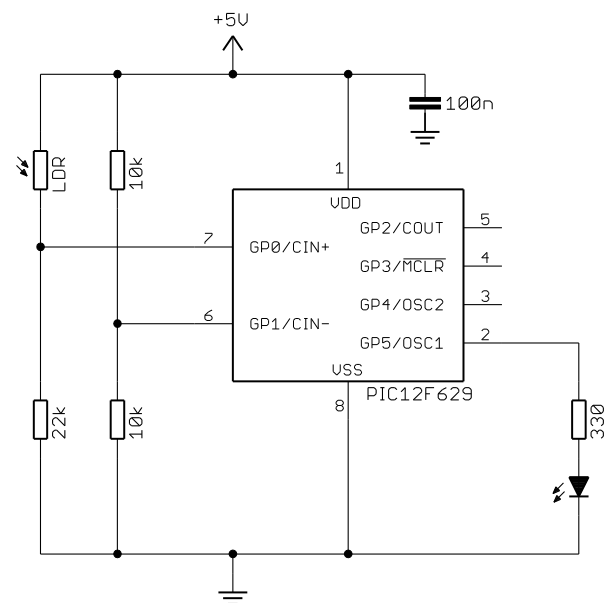


PIC12F629 Comparator Module

In [mid-range lesson 9](#), the circuit on the right, which includes a light dependent resistor (LDR, or CdS photocell), was used to demonstrate the basic operation of the PIC12F629's single comparator module.

If you have the [Gooligum training board](#), you can implement this circuit by placing a shunt across pins 2 and 3 ('LDR1'), connecting the photocell (PH1) and 22 kΩ resistor in the lower left of the board to CIN+, and closing JP3 and JP15 to enable the external $\overline{\text{MCLR}}$ pull-up resistor (for clarity, not shown here) and the LED on GP5.

The connection to CIN- (labelled 'GP/RA/RB1' on the board) is available as pin 9 on the 16-pin header. +V and GND are brought out on pins 15 and 16, respectively, making it easy to add the 10 kΩ resistors (supplied with the board), forming a voltage divider, by using the solderless breadboard.



If you have the Microchip Low Pin Count Demo Board, see [mid-range lesson 9](#) for instructions on configuring it for use with this example. Note that if you do not have a photocell available, you can still explore comparator operation by connecting the CIN+ input directly to the centre tap on the 10 kΩ potentiometer on that board.

We saw in [mid-range lesson 9](#) that, to configure the 12F629's comparator to behave like a simple standalone comparator, so that the output bit (COUT) is high if the voltage on the CIN+ input is higher than that on the CIN- input, it is necessary to select comparator mode 2 by setting the CM bits in the CMCON register to '010' and clearing the CINV bit, so that the output is not inverted:

```
movlw    0<<CINV|b'010'
                                ; select mode 2 (CM = 010):
                                ; +ref is CIN+, -ref is CIN-,
                                ; comparator on, no external output
                                ; output not inverted (CINV = 0)
                                ; -> COUT = 1 if CIN+ > CIN-

banksel  CMCON
movwf    CMCON
```

The LED attached to GP5 was turned on when the comparator output was high (COUT = 1) by using a shadow register, as follows:

```
loop    clrfs    sGPIO                ; assume COUT = 0 -> LED off
        banksel CMCON
        btfsc    CMCON,COUT          ; if comparator output high
        bsf      sGPIO,nLED          ; turn on LED

        movf     sGPIO,w              ; copy shadow to GPIO
        banksel GPIO
        movwf    GPIO

        goto     loop                ; repeat forever
```

XC8 implementation

As usual, the include files provided with the XC8 compiler define the bits comprising the CMCON register as a union of structures:

```
extern volatile union {
    struct {
        unsigned CM                :3;
        unsigned CIS                :1;
        unsigned CINV              :1;
        unsigned                  :1;
        unsigned COUT              :1;
    };
    struct {
        unsigned CM0                :1;
        unsigned CM1                :1;
        unsigned CM2                :1;
    };
} CMCONbits @ 0x019;
```

Thus, we can specify the comparator mode by writing:

```
CMCONbits.CM = 0b010;           // select comparator mode 2
```

Or, you may wish to express this in decimal:

```
CMCONbits.CM = 2;               // select comparator mode 2
```

Although it's your choice, and you will often see examples using the decimal form, the PIC data sheets specify the comparator modes in binary, so that is the format we will use here.

So to initialise the comparator, we have:

```
// configure comparator
CMCONbits.CM = 0b010;           // select mode 2:
                                // +ref is CIN+, -ref is CIN-,
                                // comparator on, no external output
CMCONbits.CINV = 0;             // output not inverted
                                // -> COUT = 1 if CIN+ > CIN-
```

Note that the remaining bits in CMCON are not being explicitly set or cleared; that is ok because in this example we don't care what values they are set to.

Alternatively, you could write this as a single statement, assigning a value to the variable CMCON:

```

CMCON = 0b00000010;      // configure comparator 1:
//-----010             select mode 2 (CM = 010):
//                        +ref is CIN+, -ref is CIN-,
//                        comparator on, no external output
//---0---                 output not inverted (CINV = 0)
//                        -> C1OUT = 1 if CIN+ > CIN-

```

This is ok, as long as you express the value in binary, so that it is obvious which bits are being set or cleared, and clearly commented, as above – although you will often see (in other people’s code) ‘CMCON = 7’ to disable the comparator.

The first form is clear and easy to maintain, and is therefore the method usually adopted in these lessons. On the other hand, a series of assignments like this requires more program memory than a single whole-register assignment. It is also no longer an *atomic* operation, where all the bits are updated at once. This can be an important consideration in some instances, but it is not relevant here.

The comparator’s output bit, **COUT**, is available as ‘CMCONbits.COUT’. For example:

```
LED = CMCONbits.COUT;    // turn on LED if comparator output high
```

With the above configuration, the LED will turn on when the photocell is illuminated.

If instead you wanted it to operate the other way, so that the LED is lit when the photocell is in darkness, you could invert the comparator output test, so that the LED is set high when **COUT** is low:

```
LED = ~CMCONbits.COUT;   // turn on LED if comparator output low
```

Alternatively, you can configure the comparator so that its output is inverted, using

```

// configure comparator
CMCONbits.CM = 0b010;      // select mode 2:
//                        +ref is CIN+, -ref is CIN-,
//                        comparator on, no external output
CMCONbits.CINV = 1;        // inverted output
// -> COUT = 1 if CIN+ < CIN-

```

Complete program

Here is the complete inverted output version of the program, for XC8:

```

/*****
*   Description:    Lesson 6, example 1b
*
*   Demonstrates use of comparator output inversion bit
*
*   Turns on LED when voltage on CIN+ < voltage on CIN-
*
*****/
*
*   Pin assignments:
*       CIN+ = voltage to be measured (e.g. pot output or LDR)
*       CIN- = threshold voltage (set by voltage divider resistors)
*       GP5  = indicator LED
*
*****/

```

```

#include <xc.h>

/***** CONFIGURATION *****/
// ext reset, no code protect, no brownout detect, no watchdog,
// power-up timer enabled, int RC clock
__CONFIG(MCLRE_ON & CP_OFF & CPD_OFF & BOREN_OFF & WDTE_OFF &
         PWRTE_OFF & FOSC_INTRCIO);

// Pin assignments
#define LED      GPIObits.GP5      // indicator LED on GP5
#define nLED     5                 // (port bit 5)

/***** MAIN PROGRAM *****/
void main()
{
    /*** Initialisation ***/

    // configure port
    TRISIO = ~(1<<nLED);           // configure LED pin (only) as an output

    // configure comparator
    CMCONbits.CM = 0b010;          // select mode 2:
                                    // +ref is CIN+, -ref is CIN-,
                                    // comparator on, no external output
    CMCONbits.CINV = 1;            // inverted output
                                    // -> COUT = 1 if CIN+ < CIN-

    /*** Main loop ***/
    for (;;)
    {
        // continually display comparator output
        LED = CMCONbits.COUT;
    }
}

```

Programmable Voltage Reference

As described in greater detail in [mid-range lesson 9](#), the PIC12F629 provides an internal voltage reference, which can be set to one of 32 available voltages, from 0 V to $0.72 \times V_{DD}$.

It is controlled by the VRCON register, as follows:

The reference voltage is set by the VR<3:0> bits and VRR, which selects a high or low voltage range:

VRR = 1 selects the low range, where $CV_{REF} = VR<3:0>/24 \times V_{DD}$.

VRR = 0 selects the high range, where $CV_{REF} = V_{DD}/4 + VR<3:0>/32 \times V_{DD}$.

With a 5 V supply, the available output range is from 0 V to 3.59 V.

Since the low and high ranges overlap, only 29 of the 32 selectable voltages are unique ($0.250 \times V_{DD}$, $0.500 \times V_{DD}$ and $0.625 \times V_{DD}$ are selectable in both ranges).

The voltage reference module can be disabled (to save power in sleep mode) by clearing the VREN bit.

In [mid-range lesson 9](#), the circuit on the right was used to demonstrate how the voltage reference module can be used instead of an external reference – allowing us to remove the 10 kΩ voltage divider resistors from the circuit.

The CIN+ input is used with the photocell to detect light levels, as before. We'll use the LED on GP5 to indicate a low level of illumination, which we'll define to be CIN+ below 1.5 V.

On the PIC12F629, CVREF can only be used as the positive comparator reference. Since the signal from the photocell in our circuit is connected to CIN+, we have to configure CIN+ as the *negative* reference.

This was done in [mid-range lesson 9](#) by:

```
; configure comparator
movlw    b'110'|1<<CIS|0<<CINV
                                ; select mode 6 (CM = 110):
                                ; +ref is CVref,
                                ; -ref is CIN+ (CIS = 1),
                                ; no external output,
                                ; comparator on
                                ; output not inverted (CINV = 0)
                                ; -> COUT = 1 if CIN+ < CVref

banksel  CMCON
movwf    CMCON
```

To generate the closest match to 1.5 V, the voltage reference was configured with VRR = 1 (selecting the low range), and VR = 7, giving a threshold of $0.292 \times 5.0 \text{ V} = 1.46 \text{ V}$ (when VDD = 5.0 V):

```
; configure voltage reference
movlw    1<<VRR|.7|1<<VREN
                                ; CVref = 0.292*Vdd (VRR = 1, VR = 7)
                                ; enable voltage reference (VREN = 1)
                                ; -> CVref = 1.5 V (if Vdd = 5.0 V)

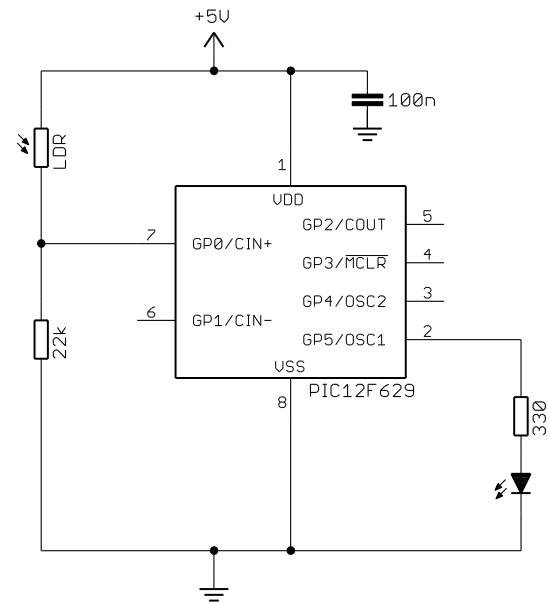
banksel  VRCON
movwf    VRCON
```

XC8 implementation

The comparator is configured in the same manner as in the last example:

```
// configure comparator
CMCONbits.CM = 0b110;           // select mode 6:
                                // +ref is CVref,
                                // no external output,
                                // comparator on
CMCONbits.CIS = 1;              // -ref is CIN+
CMCONbits.CINV = 0;             // output not inverted
                                // -> COUT = 1 if CIN+ < CVref
```

Note that, this time, we need to explicitly set the CIS bit.



Configuring the voltage reference is simply:

```
// configure voltage reference
VRCONbits.VRR = 1;           // select low range
VRCONbits.VR = 7;           // CVref = 0.292*Vdd
VRCONbits.VREN = 1;         // enable voltage reference
                             // -> CVref = 1.5 V (if Vdd = 5.0 V)
```

In the main loop, all we need do is continually copy the comparator output to the LED, as before:

```
for (;;)
{
    // continually display comparator output
    LED = CMCONbits.COUT;
}
```

External Output and Hysteresis

As was explained in [mid-range lesson 9](#), it is often desirable to add hysteresis to a comparator, to make it less sensitive to small changes in the input signal due to superimposed noise or other interference. For example, in the above examples using a photocell, you will find that the output LED flickers when the light level is close to the threshold, particularly with mains-powered artificial illumination, which varies at 50 or 60 Hz.

On the PIC12F629, the comparator output can be made available on the COUT pin (shared with GP2); we can use this to add hysteresis to the circuit.

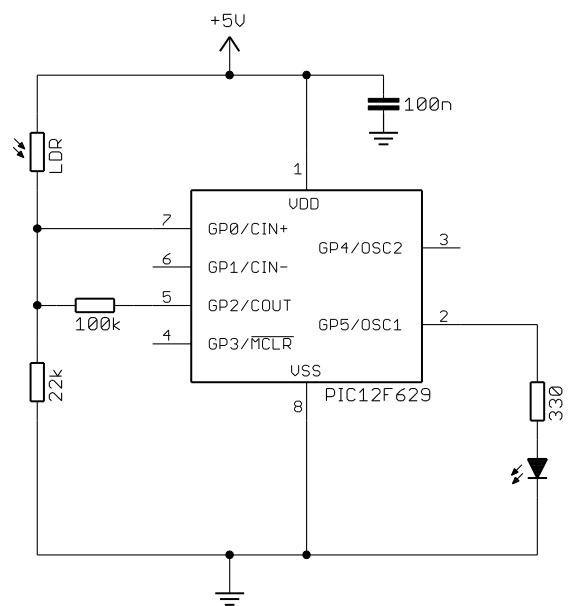
In the circuit on the right, hysteresis has been introduced by using a 100 kΩ resistor to feed some of the comparator's output, on COUT, back into the CIN+ input.

You can do this with the [Gooligum training board](#) by placing the supplied 100 kΩ resistor between pins 8 ('GP/RA/RB0') and 13 ('GP/RA/RB2') on the 16-pin header.

Or, if you are using the Microchip Low Pin Count Demo Board, you would place the feedback resistor between pins 7 and 9 on the 14-pin header.

The comparator output is enabled in comparator modes 1, 3 and 5 (binary 001, 011 and 101).

Additionally, TRISIO<2> must be cleared to allow the COUT signal to appear on the GP2/COUT pin.



Hysteresis relies on positive feedback.

As we saw in the previous example, if you are using the internal voltage reference, your external signal must be connected to the comparator's negative input.

But that's not a problem – it is possible to introduce hysteresis by feeding the comparator output into the negative input signal (i.e. the signal being measured), as long as we invert the comparator output, so that the overall feedback is still positive.

XC8 implementation

The code is essentially the same as before, but we must select comparator mode 5 (binary 101) instead of mode 6, to enable COUT:

```
// configure comparator
CMCONbits.CM = 0b101;      // select mode 5:
                             //   +ref is CVref,
                             //   external output enabled,
                             //   comparator on
CMCONbits.CIS = 1;         // -ref is CIN+
CMCONbits.CINV = 1;        // inverted output
                             // -> COUT = 1 if CIN+ > CVref,
                             //   COUT pin enabled
```

It is very important to set the CINV bit, so that the comparator output, fed via the 100 kΩ resistor into the comparator's negative input, will be inverted, providing positive feedback overall.

But before the comparator output can actually appear on the COUT pin, that pin has to be configured as an output:

```
TRISIObits.TRISIO2 = 0;    // configure COUT (GP2) as an output
```

Note that we could have done this when initialising the port, for example:

```
TRISIO = ~(1<<nLED|1<<2); // configure LED pin and COUT (GP2) as outputs
```

However, it is usually better to keep all code associated with the comparator configuration (such as enabling the COUT pin) with the rest of the comparator initialisation code; if you later want to change how the comparator is configured, you don't have to remember to make a corresponding change in some other section of initialisation code. Modular code tends to be more easily maintained.

Finally, because the comparator output is now inverted, we need to invert the display:

```
for (;;)
{
    // continually display inverse of comparator output
    LED = ~CMCONbits.COUT;
}
```

Complete program

Here is how these pieces fit together:

```
/* *****
 * Description:    Lesson 6, example 3
 *
 * Demonstrates comparator hysteresis (using COUT)
 * with programmable voltage reference
 *
 * Turns on LED when voltage on CIN+ < 1.5 V
 *
 * *****
 * Pin assignments:
 *   CIN+ = voltage to be measured (e.g. pot output or LDR)
 *   COUT = comparator output (fed back to input via resistor)
 *   GP5  = indicator LED
 *
 * ***** */

#include <xc.h>
```

```

/***** CONFIGURATION *****/
// ext reset, no code protect, no brownout detect, no watchdog,
// power-up timer enabled, int RC clock
__CONFIG(MCLRE_ON & CP_OFF & CPD_OFF & BOREN_OFF & WDTE_OFF &
        PWRTE_OFF & FOSC_INTRCIO);

// Pin assignments
#define LED      GPIObits.GP5      // indicator LED on GP5
#define nLED     5                  // (port bit 5)

/***** MAIN PROGRAM *****/
void main()
{
    /*** Initialisation ***/

    // configure port
    TRISIO = ~(1<<nLED);           // configure LED pin (only) as an output

    // configure comparator
    CMCONbits.CM = 0b101;          // select mode 5:
                                    // +ref is CVref,
                                    // external output enabled,
                                    // comparator on
    CMCONbits.CIS = 1;             // -ref is CIN+
    CMCONbits.CINV = 1;            // inverted output
                                    // -> COUT = 1 if CIN+ > CVref,
                                    // COUT pin enabled
    TRISIObits.TRISIO2 = 0;        // configure COUT (GP2) as an output

    // configure voltage reference
    VRCONbits.VRR = 1;             // select low range
    VRCONbits.VR = 7;              // CVref = 0.292*Vdd
    VRCONbits.VREN = 1;            // enable voltage reference
                                    // -> CVref = 1.5 V (if Vdd = 5.0 V)

    /*** Main loop ***/
    for (;;)
    {
        // continually display inverse of comparator output
        LED = ~CMCONbits.COUT;
    }
}

```

Comparator Interrupts

The comparator module can be selected as an interrupt source. This allows us to respond immediately to a change in the comparator output (i.e. the analog input crossing a threshold), without having to continually poll COUT, as we were doing above. Instead, you can choose to generate an interrupt whenever COUT changes, and handle the “comparator change” event in your interrupt service routine (ISR) – much as we did for pin change interrupts in [lesson 4](#).

Like all interrupts (see [lesson 3](#)), the comparator interrupt has an associated enable bit, which has to be set to allow the comparator to trigger interrupts, and an interrupt flag bit, which is set whenever the comparator’s output changes. As we saw in [mid-range lesson 9](#), the comparator is considered to be a *peripheral* (not part of the PIC core), and its interrupt bits are held in peripheral interrupt registers.

Peripheral interrupts on the PIC12F629 are enabled by setting the PEIE (peripheral interrupt enable) bit in the INTCON register.

The comparator interrupt is enabled by setting the CMIE bit in the PIE1 (peripheral interrupt enable 1) register.

And as we have seen, the GIE bit, in the INTCON register, must be set to globally enable interrupts.

So, to enable comparator interrupts, we must set CMIE, PEIE and GIE.

The comparator interrupt flag, CMIF, is located in PIR1 (peripheral interrupt register 1). It is set whenever the comparator output (COUT) has changed since the last time CMCON was read or written.

As with other interrupt sources, the comparator interrupt flag is active, whether or not comparator interrupts are enabled. This means that you could, in principle, poll CMIF to detect changes in COUT – although it would normally make more sense to simply poll COUT directly.

In the interrupt service routine, you should read or write CMCON, to clear any existing *mismatch condition*, to allow you to successfully clear the CMIF flag. And to avoid false triggering (an interrupt occurring because of a previous change), you should also read or write CMCON and clear CMIF, immediately before enabling the comparator interrupt.

To demonstrate this, we can re-implement the last example, using interrupts.

Instead of polling COUT, we'll set the comparator module to trigger an interrupt whenever the input (light level) passes through the threshold, and have the ISR turn on or off the LED.

This then leaves the main program code free to perform other tasks – although in this example, all we will do is update the LED display.

We can keep the comparator and voltage reference configuration the same as before. It's a good idea to leave the hysteresis in place, as this minimises the number of comparator transitions – limiting the number of times the interrupt will be triggered.

In [mid-range lesson 9](#), the comparator interrupt was enabled by:

```
movlw    1<<PEIE|1<<GIE      ; enable peripheral and global interrupts
movwf    INTCON
banksel  CMCON                ; enable comparator interrupt:
movf     CMCON,w              ; read CMCON to clear mismatch
banksel  PIR1
bcf      PIR1,CMIF            ; clear interrupt flag
banksel  PIE1
bsf      PIE1,CMIE            ; set enable bit
```

In the interrupt handler, the comparator mismatch condition was removed first, allowing the interrupt flag to be cleared:

```
banksel  CMCON
movf     CMCON,w              ; clear mismatch condition
banksel  PIR1
bcf      PIR1,CMIF            ; clear interrupt flag
```

Since the comparator interrupt is triggered by any comparator output change, we can't know whether the interrupt was caused by the comparator input going high or low.

So, we checked the value of COUT, and set the LED output accordingly, using:

```
; test for low comparator input and display
clrfs    sGPIO                ; assume COUT = 0 -> LED off
banksel  CMCON
btfss    CMCON,COUT            ; if comparator output low (CIN+ < 1.5 V)
bsf      sGPIO,nLED           ; turn on LED (using shadow register)
```

The main loop then simply copied the shadow register to GPIO.

XC8 implementation

Implementing these steps using XC8 is quite straightforward; the techniques are similar to those used to in the interrupt-on-change example in [lesson 4](#).

First we enable the peripheral and global interrupts:

```
INTCONbits.PEIE = 1;          // enable peripheral interrupts
ei();                       // enable global interrupts
```

Recall that, before enabling the comparator interrupt, we should read CMCON, and clear CMIF, in case a comparator change has occurred since execution began, to avoid triggering the comparator interrupt the moment it is enabled³.

We saw in [lesson 4](#) that, given an expression consisting only of a register name, the XC8 compiler will generate an instruction which does nothing other than read that register.

So, to read CMCON, we can simply write:

```
CMCON;                       // read CMCON to clear mismatch
```

and then to clear CMIF and enable the comparator interrupt, we have:

```
PIR1bits.CMIF = 0;           // clear interrupt flag
PIELbits.CMIE = 1;           // set enable bit
```

The interrupt initialisation is done in this order, with peripheral and global interrupts being enabled first, to limit the time between clearing CMCON and enabling the comparator interrupt. But of course, a few microseconds won't really matter, so in practice you could instead enable the comparator interrupt (and any other interrupt sources) before enabling peripheral and global interrupts last:

```
CMCON;                       // enable comparator interrupt:
PIR1bits.CMIF = 0;           // read CMCON to clear mismatch
PIELbits.CMIE = 1;           // clear interrupt flag
INTCONbits.PEIE = 1;         // set enable bit
ei();                       // enable peripheral interrupts
ei();                       // enable global interrupts
```

If you take this approach and enable global interrupts last, all your interrupt sources will become active at once. This may or may not represent a problem, but it's something to consider. In this instance, with only one interrupt source enabled, it makes no real difference.

³ In this example, there would be no problem if the comparator interrupt was triggered by a change which had happened before it was enabled, but that may not be true in other applications.

In the interrupt handler, we explicitly clear the comparator mismatch (instead of relying on later statements within the handler routine), and clear the interrupt flag:

```
CMCON;                // read CMCON to clear mismatch condition
PIR1bits.CMIF = 0;    // clear interrupt flag
```

We can then turn the LED on or off, depending on whether the comparator output has gone high or low.

In the assembler example, this was done by first clearing the shadow register and then setting the bit corresponding to the LED only if COUT = 0 (implying that CIN+ < 1.5 V).

But in 'C', we can express this very simply as:

```
sLED = ~CMCONbits.COUT;    // display inverse of comparator output
```

Then in the main loop, we do nothing but copy the shadow register to the port:

```
for (;;)
{
    // continually copy shadow GPIO to port
    GPIO = sGPIO.port;
}
```

Complete program

Here is how these code fragments come together:

```

/*****
 *
 * Description:    Lesson 6, example 4
 *
 * Demonstrates use of comparator interrupt
 * (assumes hysteresis is used to reduce triggering)
 *
 * Turns on LED when voltage on CIN+ < 1.5 V
 *
 *****/
 *
 * Pin assignments:
 *     CIN+ = voltage to be measured (e.g. pot output or LDR)
 *     COUT = comparator output (fed back to input via resistor)
 *     GP5  = indicator LED
 *
 *****/

#include <xc.h>
#include <stdint.h>

/***** CONFIGURATION *****/
// ext reset, no code protect, no brownout detect, no watchdog,
// power-up timer enabled, int RC clock
__CONFIG(MCLRE_ON & CP_OFF & CPD_OFF & BOREN_OFF & WDTE_OFF &
        PWRTF_OFF & FOSC_INTRCIO);

// Pin assignments
#define sLED    sGPIO.GP5        // indicator LED (shadow)

/***** GLOBAL VARIABLES *****/
volatile union {                // shadow copy of GPIO
```

```

uint8_t      port;
struct {
    unsigned   GP0      : 1;
    unsigned   GP1      : 1;
    unsigned   GP2      : 1;
    unsigned   GP3      : 1;
    unsigned   GP4      : 1;
    unsigned   GP5      : 1;
};
} sGPIO;

/***** MAIN PROGRAM *****/
void main()
{
    /*** Initialisation ***/

    // configure port
    GPIO = 0;                                // start with LED off
    sGPIO.port = 0;                          // update shadow
    TRISIO = ~(1<<5);                       // configure GP5 (only) as an output

    // configure comparator
    CMCONbits.CM = 0b101;                   // select mode 5:
                                           // +ref is CVref,
                                           // external output enabled,
                                           // comparator on
    CMCONbits.CIS = 1;                      // -ref is CIN+
    CMCONbits.CINV = 1;                    // inverted output
                                           // -> COUT = 1 if CIN+ > CVref,
                                           // COUT pin enabled
    TRISIObits.TRISIO2 = 0;                // configure COUT (GP2) as an output

    // configure voltage reference
    VRCONbits.VRR = 1;                     // select low range
    VRCONbits.VR = 7;                      // CVref = 0.292*Vdd
    VRCONbits.VREN = 1;                   // enable voltage reference
                                           // -> CVref = 1.5 V (if Vdd = 5.0 V)

    // enable interrupts
    INTCONbits.PEIE = 1;                   // enable peripheral interrupts
    ei();                                  // enable global interrupts
                                           // enable comparator interrupt:
    CMCON;                                // read CMCON to clear mismatch
    PIR1bits.CMIF = 0;                    // clear interrupt flag
    PIE1bits.CMIE = 1;                    // set enable bit

    /*** Main loop ***/
    for (;;)
    {
        // continually copy shadow GPIO to port
        GPIO = sGPIO.port;
    }
}

/***** INTERRUPT SERVICE ROUTINE *****/
void interrupt_isr(void)
{
    /*** Service comparator interrupt

```

```

//
//   Triggered on any comparator output change,
//   caused by comparator input crossing 1.5 V threshold
//
//   Turns on indicator LED if CIN+ < 1.5 V
//
//   (only port change interrupts are enabled)
//
CMCON;                // read CMCON to clear mismatch condition
PIR1bits.CMIF = 0;    // clear interrupt flag

// turn on LED if CIN+ < 1.5 V
sLED = ~CMCONbits.COUT; // display inverse of comparator output
}

```

Wake-up on Comparator Change

The comparator module does not depend on the PIC's oscillator, and so can remain active while the device is in sleep mode⁴. This means that the comparator can be used to wake the PIC from sleep mode, as can any interrupt source, which is active in sleep mode – as explained in [lesson 4](#) and in [mid-range lesson 9](#).

This is useful for conserving power while waiting for a signal change from a sensor.

To enable wake-up on comparator change, simply enable the comparator interrupt, by setting the CMIE and PEIE enable bits.

If you only want to use the comparator to wake the PIC from sleep mode, and do not want to use interrupts, leave interrupts disabled, by clearing GIE.

This was demonstrated in [mid-range lesson 9](#), using the previous circuit, by configuring the PIC to wake on comparator change, and then go to sleep. When the input signal crosses the threshold level, the comparator output changes, and the PIC wakes. To indicate this we turned on the LED for one second. The PIC then went back to sleep, to wait until the next comparator change.

In the assembler example, after configuring the comparator and voltage reference as usual, a delay of 10 ms was added to allow them to settle before entering sleep mode. This is a good idea because signal levels can take a while to settle after power-on, and if the comparator output was changing while going into sleep mode, the PIC would immediately wake and the LED would flash – a false trigger.

XC8 implementation

To enable wake on comparator change, we need to enable the comparator interrupt, but not global interrupts (since we're not actually using interrupts here; we're only using an interrupt source to wake the device):

```

// enable comparator interrupt (for wake on change)
INTCONbits.PEIE = 1;    // enable peripheral interrupts
PIE1bits.CMIE = 1;     // enable comparator interrupt

```

Within the main loop, we can start by turning off the LED, because we don't want it lit on start-up:

```

// turn off LED
LED = 0;

```

⁴ If you are not using the comparator to wake the device, you should turn it off before entering sleep mode, to conserve power.

Before entering sleep mode, it is very important to clear any existing comparator mismatch condition, as well as CMIF:

```
// go into standby (low power) mode
CMCON;           // read CMCON to clear comparator mismatch
PIR1bits.CMIF = 0; // clear comparator interrupt flag
SLEEP();         // enter sleep mode
```

The PIC will now sleep until it is woken by a comparator change, which we wish to show by lighting the LED for one second:

```
// turn on LED for 1 second
LED = 1;           // turn on LED
__delay_ms(1000);  // delay 1 sec
```

Complete program

Here is the complete XC8 version of the comparator wakeup example program:

```

/*****
*
*   Description:      Lesson 6, example 5
*
*   Demonstrates wake-up on comparator change
*
*   Turns on LED for 1 sec when comparator output changes
*   then sleeps until the next change
*   (threshold is 1.5 V, set by programmable voltage ref,
*    with hysteresis used to reduce sensitivity to small changes)
*
*****/
*
*   Pin assignments:
*   CIN+ = voltage to be measured (e.g. pot output or LDR)
*   COUT = comparator output (fed back to input via resistor)
*   GP5  = indicator LED
*
*****/

#include <xc.h>

#define _XTAL_FREQ    4000000    // oscillator frequency for _delay()

/***** CONFIGURATION *****/
// ext reset, no code protect, no brownout detect, no watchdog,
// power-up timer enabled, int RC clock
__CONFIG(MCLRE_ON & CP_OFF & CPD_OFF & BOREN_OFF & WDTE_OFF &
         PWRTE_OFF & FOSC_INTRCIO);

// Pin assignments
#define LED      GPIObits.GP5    // indicator LED
#define nLED     5               // (port bit 5)

/***** MAIN PROGRAM *****/
void main()
{
    /*** Initialisation ***/

```

```

// configure port
TRISIO = ~(1<<nLED);           // configure LED pin (only) as an output

// configure comparator
CMCONbits.CM = 0b101;          // select mode 5:
                                //   +ref is CVref,
                                //   external output enabled,
                                //   comparator on
CMCONbits.CIS = 1;              // -ref is CIN+
CMCONbits.CINV = 1;             // inverted output
                                // -> COUT = 1 if CIN+ > CVref,
                                //   COUT pin enabled
TRISIObits.TRISIO2 = 0;        // configure COUT (GP2) as an output

// configure voltage reference
VRCONbits.VRR = 1;              // select low range
VRCONbits.VR = 7;               //   CVref = 0.292*Vdd
VRCONbits.VREN = 1;             // enable voltage reference
                                // -> CVref = 1.5 V (if Vdd = 5.0 V)

// delay 10 ms to allow comparator and voltage ref to settle
__delay_ms(10);

// enable comparator interrupt (for wake on change)
INTCONbits.PEIE = 1;           // enable peripheral interrupts
PIELbits.CMIE = 1;             // enable comparator interrupt

/**** Main loop ****/
for (;;)
{
    // turn off LED
    LED = 0;

    // go into standby (low power) mode
    CMCON;                      // read CMCON to clear comparator mismatch
    PIR1bits.CMIF = 0;          // clear comparator interrupt flag
    SLEEP();                    // enter sleep mode

    // turn on LED for 1 second
    LED = 1;                    // turn on LED
    __delay_ms(1000);           // delay 1 sec
}
}

```

Additional Single-Comparator Examples

Comparing a signal against a range of thresholds

It is sometimes necessary to check that a signal is within certain limits; a band of allowed values – and have to take action if the signal is too high or too low.

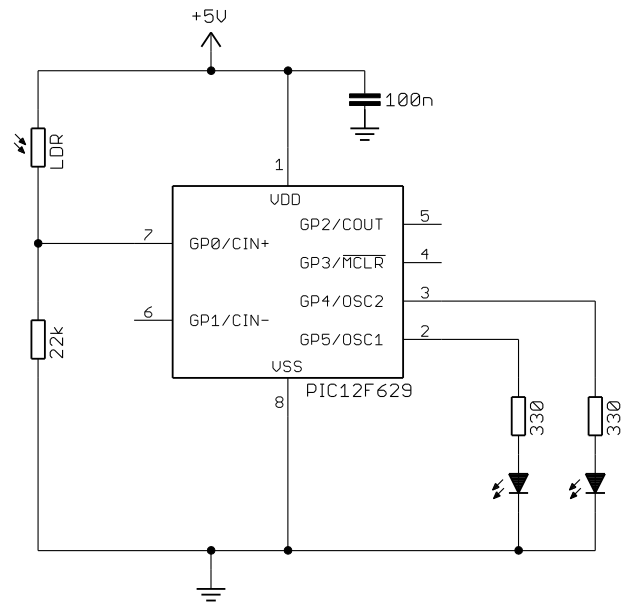
Or you may have a number of thresholds you need to compare the signal against, perhaps defining “warning”, “danger” and “automatic shut-down”.

One way to achieve this is to use the programmable voltage reference – setting it to each threshold in turn.

The circuit on the right was used in [mid-range lesson 9](#) to demonstrate this. The LED on GP5 is used to indicate a low level of illumination, while the LED on GP4 indicates bright light. When neither LED is lit, the light level will be in the middle; not too dim or too bright.

To implement this circuit using the [Gooligum training board](#), you should remove the feedback resistor (used in the previous examples), but leave the shunts across pins 2 and 3 ('LDR1') of JP24, connecting the photocell (PH1) to CIN+, and in JP3 and JP15, enabling the external $\overline{\text{MCLR}}$ pull-up resistor (for clarity, not shown here) and the LED on GP5. You should also close JP14, to enable the LED on GP4.

If you have the Microchip Low Pin Count Demo Board, you can connect the LED labelled 'DS3' to GP4 by adding a link between pins 2 and 12 on the 14-pin header.



To test whether the input is within limits, the programmable voltage reference is first configured to generate the “low” threshold voltage, the input is compared with this low level, and then the voltage reference reconfigured to generate the “high” threshold and the input compared with this high level.

This process could be extended to multiple input thresholds, by configuring the voltage reference to generate each threshold in turn. However, if you wish to test against more than a few threshold levels, you would probably be better off using an analog-to-digital converter (described in [lesson 8](#)).

This example uses 2.0 V as the “low” threshold and 3.0 V as the “high” threshold, but, since the reference is programmable, you can always choose your own levels!

XC8 implementation

The comparator is configured to compare CIN+ with CVREF, as we have done before:

```
CMCONbits.CM = 0b110;    // select mode 6:
                          // +ref is CVref,
                          // no external output,
                          // comparator on
CMCONbits.CIS = 1;       // -ref is CIN+
CMCONbits.CINV = 1;      // inverted output
                          // -> COUT = 1 if CIN+ > CVref
```

We can enable the voltage reference in our initialisation routine:

```
// enable voltage reference
VRCONbits.VREN = 1;
```

And then within the main loop we can configure it to generate approximately 2.0 V, by:

```
VRCONbits.VRR = 0;       // select high range
VRCONbits.VR = 5;        // CVref = 0.406*Vdd
                          // -> CVref = 2.03 V (if Vdd = 5.0 V)
```

And then approximately 3.0 V by:

```
VRCONbits.VRR = 0;       // select high range
VRCONbits.VR = 11;       // CVref = 0.594*Vdd
                          // -> CVref = 2.97 V (if Vdd = 5.0 V)
```

After changing the voltage reference, it can take a little while for it to settle and generate a stable voltage. According to table 12-7 in the PIC12F629/675 data sheet, this settling time can be up to 10 μ s.

Therefore, we should insert a 10 μ s delay after configuring the voltage reference, before reading the comparator output.

This can be done using the ‘`__delay_us()`’ macro built into XC8:

```
__delay_us(10);           // wait 10 us to settle
```

Complete program

Here is the complete XC8 version of the “test that a signal is within limits” example:

```

/*****
 *
 *   Description:      Lesson 6, example 6
 *
 *   Demonstrates use of programmable voltage reference
 *   to test that a signal is within limits
 *
 *   Turns on Low LED  when CIN+ < 2.0 V (low light level)
 *   or High LED when CIN+ > 3.0 V (high light level)
 *
 *****/
 *
 *   Pin assignments:
 *   CIN+ = voltage to be measured (e.g. pot output or LDR)
 *   GP5  = "Low" LED
 *   GP4  = "High" LED
 *
 *****/

#include <xc.h>
#include <stdint.h>

#define _XTAL_FREQ    4000000    // oscillator frequency for _delay()

/***** CONFIGURATION *****/
// ext reset, no code protect, no brownout detect, no watchdog,
// power-up timer enabled, int RC clock
__CONFIG(MCLRE_ON & CP_OFF & CPD_OFF & BOREN_OFF & WDTE_OFF &
         PWRTE_OFF & FOSC_INTRCIO);

// Pin assignments
#define sLO      sGPIO.GP5          // "Low" LED (shadow)
#define sHI      sGPIO.GP4          // "High" LED (shadow)

/***** GLOBAL VARIABLES *****/
volatile union {                  // shadow copy of GPIO
    uint8_t      port;
    struct {
        unsigned GP0      : 1;
        unsigned GP1      : 1;
        unsigned GP2      : 1;
        unsigned GP3      : 1;
        unsigned GP4      : 1;
        unsigned GP5      : 1;
    };
} sGPIO;
```

```

/***** MAIN PROGRAM *****/
void main()
{
    /*** Initialisation ***/

    // configure port
    TRISIO = 0b001111;           // configure GP4 and GP5 as outputs

    // configure comparator
    CMCONbits.CM = 0b110;        // select mode 6:
                                //   +ref is CVref,
                                //   no external output,
                                //   comparator on
    CMCONbits.CIS = 1;           // -ref is CIN+
    CMCONbits.CINV = 1;          // inverted output
                                // -> COUT = 1 if CIN+ > CVref

    // enable voltage reference
    VRCONbits.VREN = 1;

    /*** Main loop ***/
    for (;;)
    {
        // Test for low illumination
        // set low input threshold
        VRCONbits.VRR = 0;        // select high range
        VRCONbits.VR = 5;         //   CVref = 0.406*Vdd
                                // -> CVref = 2.03 V (if Vdd = 5.0 V)
        __delay_us(10);           // wait 10 us to settle

        // compare with input
        sLO = ~CMCONbits.COUT;     // turn on Low LED if CIN+ < CVref

        // Test for high illumination
        // set high input threshold
        VRCONbits.VRR = 0;        // select high range
        VRCONbits.VR = 11;        //   CVref = 0.594*Vdd
                                // -> CVref = 2.97 V (if Vdd = 5.0 V)
        __delay_us(10);           // wait 10 us to settle

        // compare with input
        sHI = CMCONbits.COUT;     // turn on High LED if CIN+ > CVref

        // Display test results
        GPIO = sGPIO.port;        // copy shadow GPIO to port
    }
}

```

Testing two independent inputs

Suppose that you need to compare two input signals (say, light level in two locations) against a common reference. You could use two comparators, with each input connected to a different comparator, and the common reference voltage connected to both. But, what if you only have a single comparator available, as in the PIC12F629?

Recall that, in comparator modes 5 and 6 (binary 101 and 110), the CIS bit in the CMCON register selects whether the CIN+ or CIN- pin is connected to the comparator's negative input.

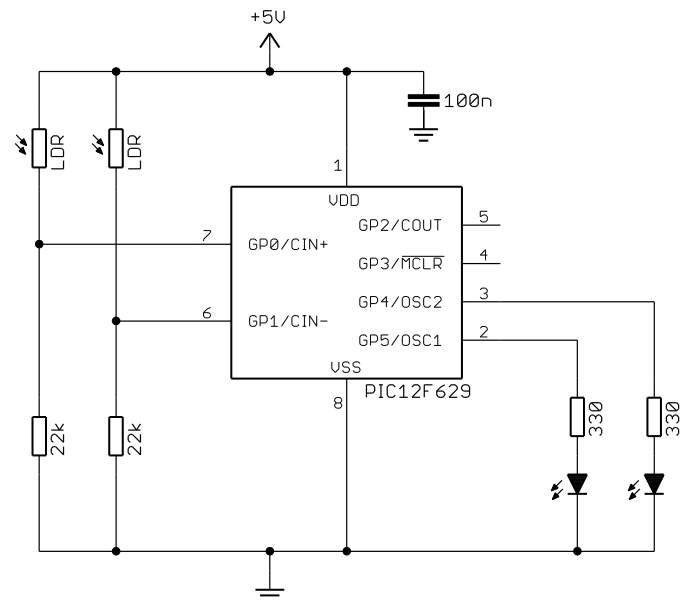
In these modes, the negative input is said to be *multiplexed*, making it easy to switch between the CIN+ and CIN- pins, on the fly.

This means that we can select CIN+ as an input, perform a comparison with the internal reference voltage, and then select CIN- and repeat the comparison⁵.

This was demonstrated in [mid-range lesson 9](#), using the circuit on the right (the reset switch and pull-up are omitted for clarity), where an additional photocell was added to the circuit used in the last example.

If you are using the [Gooligum training board](#), you can keep the board set up as before, with a shunt across pins 2 and 3 ('LDR1') of JP24, and JP3, JP14 and JP15 closed. You should also place a shunt in position 3 ('C1IN-') of JP25, to connect photocell PH2 to CIN-.

The LED on GP4 is lit whenever the CIN+ input is above 2.5 V (indicating bright light), while the LED on GP5 indicates bright light on the photocell connected to CIN-.



XC8 implementation

When configuring the comparator, there is no need to explicitly initialise CIS, since we'll be using it to switch between inputs, later:

```
CMCONbits.CM = 0b110;           // select mode 6:
                                // +ref is CVref,
                                // -ref is CIN+ or CIN- (selected by CIS)
                                // no external output,
                                // comparator on
CMCONbits.CINV = 1;             // inverted output
                                // -> COOUT = 1 if -ref > CVref
```

Within the main loop, we can then select each input in turn:

```
// test input 1
CMCONbits.CIS = 1;              // select CIN+ pin as -ref
```

After changing the comparator's configuration, or mode, it can take a while for it to settle and generate a valid output. According to table 12-6 in the PIC629/675 data sheet, this ("Comparator Mode Change to Output Valid") can take up to 10 μ s. By coincidence, that's the same as the voltage reference settling time, discussed in the last example, although they are actually unrelated.

⁵ Since the voltage reference is programmable, you could setup a different threshold for each input, by reprogramming the reference voltage each time you switch between inputs – but for clarity we won't do that in this example.

Therefore, we need to add a 10 μ s delay after selecting a new input, before reading the comparator output, using:

```
__delay_us(10);          // wait 10 us to settle
```

If we copy the comparator output bit to the shadow LED bit, the LED will be lit only if the CIN+ input is higher than the threshold:

```
sLED1 = CMCONbits.COUT;    // turn on LED1 if CIN+ > CVref
```

We can then do the same again, selecting the other comparator input:

```
// test input 2
CMCONbits.CIS = 0;          // select CIN- pin as -ref
__delay_us(10);             // wait 10 us to settle
sLED2 = CMCONbits.COUT;     // turn on LED2 if CIN- > CVref
```

And, since this code has updated a shadow variable, instead of writing directly to GPIO, we need to display any updates by writing the shadow copy to GPIO:

```
// display test results
GPIO = sGPIO.port;          // copy shadow GPIO to port
```

Complete program

Here is the full “two inputs with a common programmed voltage reference” program, showing how these fragments fit together:

```

/*****
*
*   Description:      Lesson 6, example 7
*
*   Demonstrates comparator input multiplexing
*
*   Turns on: LED 1 when CIN+ > 2.5 V
*               and LED 2 when CIN- > 2.5 V
*
*****/
*
*   Pin assignments:
*       CIN+ = input 1 (LDR/resistor divider)
*       CIN- = input 2 (LDR/resistor divider)
*       GP4  = indicator LED 1
*       GP5  = indicator LED 2
*
*****/

#include <xc.h>
#include <stdint.h>

#define _XTAL_FREQ    4000000    // oscillator frequency for __delay()

/***** CONFIGURATION *****/
// ext reset, no code protect, no brownout detect, no watchdog,
// power-up timer enabled, int RC clock
__CONFIG(MCLRE_ON & CP_OFF & CPD_OFF & BOREN_OFF & WDTE_OFF &
         PWRTE_OFF & FOSC_INTRCIO);

```

```

// Pin assignments
#define sLED1    sGPIO.GP4        // indicator LED 1
#define sLED2    sGPIO.GP5        // indicator LED 2

/***** GLOBAL VARIABLES *****/
volatile union {                  // shadow copy of GPIO
    uint8_t      port;
    struct {
        unsigned GP0      : 1;
        unsigned GP1      : 1;
        unsigned GP2      : 1;
        unsigned GP3      : 1;
        unsigned GP4      : 1;
        unsigned GP5      : 1;
    };
} sGPIO;

/***** MAIN PROGRAM *****/
void main()
{
    /*** Initialisation ***/

    // configure port
    TRISIO = 0b001111;           // configure GP4 and GP5 as outputs

    // configure comparator
    CMCONbits.CM = 0b110;        // select mode 6:
                                //   +ref is CVref,
                                //   -ref is CIN+ or CIN- (selected by CIS)
                                //   no external output,
                                //   comparator on
    CMCONbits.CINV = 1;          // inverted output
                                // -> COUT = 1 if -ref > CVref

    // configure voltage reference
    VRCONbits.VRR = 1;           // select low range
    VRCONbits.VR = 12;           //   CVref = 0.500*Vdd
    VRCONbits.VREN = 1;          // enable voltage reference
                                // -> CVref = 2.5 V (if Vdd = 5.0 V)

    /*** Main loop ***/
    for (;;)
    {
        // test input 1
        CMCONbits.CIS = 1;        // select CIN+ pin as -ref
        __delay_us(10);           // wait 10 us to settle
        sLED1 = CMCONbits.COUT;    // turn on LED1 if CIN+ > CVref

        // test input 2
        CMCONbits.CIS = 0;        // select CIN- pin as -ref
        __delay_us(10);           // wait 10 us to settle
        sLED2 = CMCONbits.COUT;    // turn on LED2 if CIN- > CVref

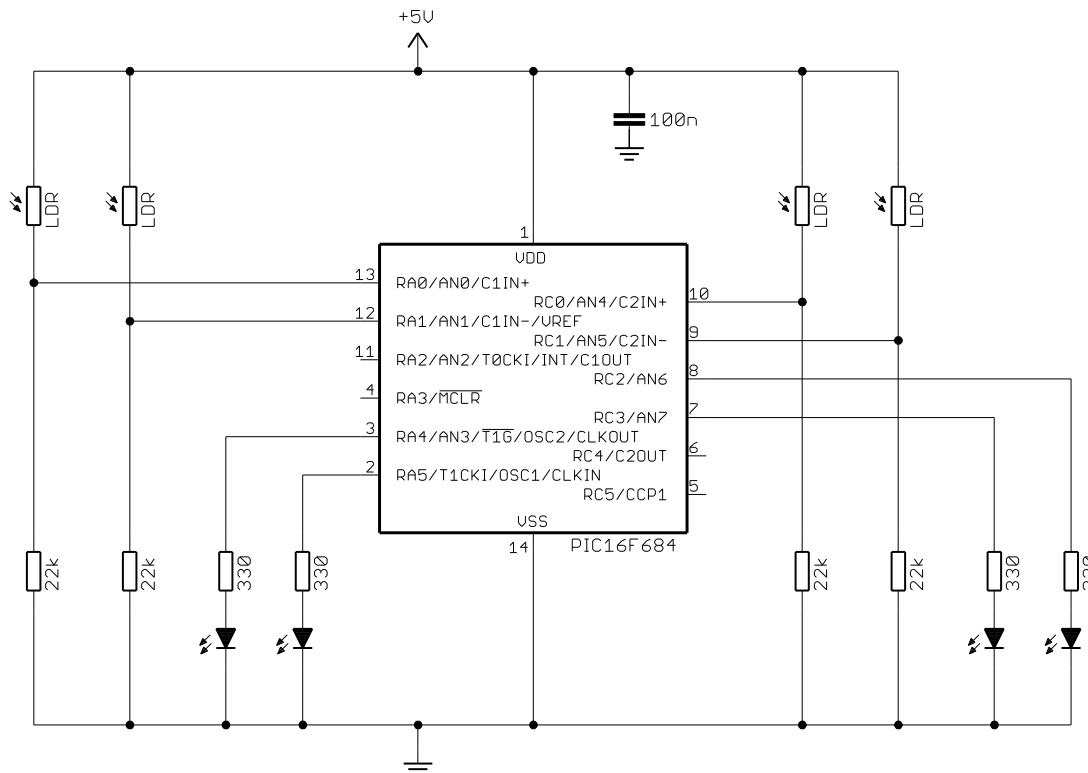
        // display test results
        GPIO = sGPIO.port;        // copy shadow GPIO to port
    }
}

```

PIC16F684 Dual Comparator Module

The PIC16F684, introduced in [mid-range lesson 10](#), includes a dual comparator module that is essentially an extended version of the PIC12F629's single comparator; see [mid-range lesson 11](#) for details.

In [mid-range lesson 11](#), the circuit below was used to demonstrate the basic operation of the PIC16F684's dual comparator module, by showing how it can be used with four inputs (photocells, in this case).



To build this circuit with the [Gooligum training board](#), use the solderless breadboard to connect the supplied photocells and 22 k Ω resistors to C2IN+ (available on pin 10, 'RC0', on the 16-pin header), C2IN- (pin 12, 'RC1'), VDD (pin 15, '+V) and ground (pin 16, 'GND'), as shown in the circuit diagram. You also need to place a shunt across pins 2 and 3 ('LDR1') of JP24 to connect photocell PH1 to C1IN+, and a shunt in position 3 ('C1IN-') of JP25, to connect photocell PH2 to C1IN-.

Finally, close jumpers JP14, JP15, JP18 and JP19 to connect LEDs to RA4, RA5, RC2 and RC3.

As in the last example, the programmable voltage reference is used to generate a common threshold level of 2.5 V. When any of the comparator inputs is higher than this threshold (corresponding to bright light), the corresponding LED is turned on.

XC8 implementation

To use the programmable voltage reference on the 16F684, we must choose comparator mode 2 (010).

As in the last example, when configuring the comparator, there is no need to initialise CIS, since we'll be using it to switch between inputs, later:

```
// configure comparators
CMCON0bits.CM = 0b010;    // select mode 2:
                           // C1 +ref is CVref,
                           // C1 -ref is C1IN+ or C1IN- (selected by CIS),
                           // C2 +ref is CVref,
                           // C2 -ref is C2IN+ or C2IN- (selected by CIS),
                           // no external outputs,
                           // both comparators on
CMCON0bits.C1INV = 1;     // C1 output inverted
                           // -> C1OUT = 1 if C1 -ref > CVref
CMCON0bits.C2INV = 1;     // C2 output inverted
                           // -> C2OUT = 1 if C2 -ref > CVref
```

The voltage reference is configured as before:

```
// configure voltage reference
VRCONbits.VRR = 1;        // select low range
VRCONbits.VR = 12;        // CVref = 0.500*Vdd
VRCONbits.VREN = 1;       // enable voltage reference
                           // -> CVref = 2.5 V (if Vdd = 5.0 V)
```

The tests in the main loop are much the same as in the previous example, except that changing the value of CIS now selects a pair of inputs, instead of a single input, because it affects both comparators:

```
// test inputs 1 (C1IN+) and 2 (C2IN+)
CMCON0bits.CIS = 1;        // select C1IN+ and C2IN+ pins as -refs
__delay_us(10);           // wait 10us to settle
sLED1 = CMCON0bits.C1OUT;  // turn on LED1 if C1IN+ > CVref
sLED2 = CMCON0bits.C2OUT;  // turn on LED2 if C2IN+ > CVref

// test inputs 3 (C2IN-) and 4 (C1IN-)
CMCON0bits.CIS = 0;        // select C1IN- and C2IN- pins as -refs
__delay_us(10);           // wait 10us to settle
sLED3 = CMCON0bits.C2OUT;  // turn on LED2 if C2IN- > CVref
sLED4 = CMCON0bits.C1OUT;  // turn on LED3 if C1IN- > CVref
```

Complete program

The structure of this “four inputs with a common programmed voltage reference” example is very similar to the “two inputs” program in the last example, but here is the full XC8 version, for reference:

```
/******
 *
 * Description:    Lesson 6, example 8
 *
 * Demonstrates dual comparator input multiplexing
 *
 * Turns on: LED 1 when C1IN+ > 2.5 V,
 *           LED 2 when C1IN- > 2.5 V,
 *           LED 3 when C2IN+ > 2.5 V,
 *           and LED 4 when C2IN- > 2.5 V
 *
 *****/
```

```

*   Pin assignments:
*   C1IN+ = input 1 (LDR/resistor divider)
*   C2IN+ = input 2 (LDR/resistor divider)
*   C2IN- = input 3 (LDR/resistor divider)
*   C1IN- = input 4 (LDR/resistor divider)
*   RC3   = indicator LED 1
*   RC2   = indicator LED 2
*   RA4   = indicator LED 3
*   RA5   = indicator LED 4
*
*****/

#include <xc.h>
#include <stdint.h>

#define _XTAL_FREQ 4000000 // oscillator frequency for _delay()

/***** CONFIGURATION *****/
// ext reset, no code or data protect, no brownout detect,
// no watchdog, power-up timer enabled, int clock with I/O,
// no failsafe clock monitor, two-speed start-up disabled
__CONFIG(MCLRE_ON & CP_OFF & CPD_OFF & BOREN_OFF &
        WDTE_OFF & PWRTE_ON & FOSC_INTOSCIO &
        FCMEN_OFF & IESO_OFF);

// Pin assignments
#define sLED1  SPORTC.RC3 // indicator LED 1 (shadow)
#define sLED2  SPORTC.RC2 // indicator LED 2 (shadow)
#define sLED3  SPORTA.RA4 // indicator LED 3 (shadow)
#define sLED4  SPORTA.RA5 // indicator LED 4 (shadow)

/***** GLOBAL VARIABLES *****/
union { // shadow copy of PORTA
    uint8_t RA;
    struct {
        unsigned RA0 : 1;
        unsigned RA1 : 1;
        unsigned RA2 : 1;
        unsigned RA3 : 1;
        unsigned RA4 : 1;
        unsigned RA5 : 1;
    };
} SPORTA;

union { // shadow copy of PORTC
    uint8_t RC;
    struct {
        unsigned RC0 : 1;
        unsigned RC1 : 1;
        unsigned RC2 : 1;
        unsigned RC3 : 1;
        unsigned RC4 : 1;
        unsigned RC5 : 1;
    };
} SPORTC;

/***** MAIN PROGRAM *****/
void main()

```

```

{
    /*** Initialisation ***/

    // configure ports
    TRISA = 0b000011;           // configure RA0/C1IN+ and RA1/C1IN- as inputs,
                                // rest of PORTA as outputs
    TRISC = 0b000011;           // configure RC0/C2IN+ and RC1/C2IN- as inputs,
                                // rest of PORTC as outputs

    // configure comparators
    CMCON0bits.CM = 0b010;      // select mode 2:
                                // C1 +ref is CVref,
                                // C1 -ref is C1IN+ or C1IN- (selected by CIS),
                                // C2 +ref is CVref,
                                // C2 -ref is C2IN+ or C2IN- (selected by CIS),
                                // no external outputs,
                                // both comparators on
    CMCON0bits.C1INV = 1;        // C1 output inverted
                                // -> C1OUT = 1 if C1 -ref > CVref
    CMCON0bits.C2INV = 1;        // C2 output inverted
                                // -> C2OUT = 1 if C2 -ref > CVref

    // configure voltage reference
    VRCONbits.VRR = 1;          // select low range
    VRCONbits.VR = 12;          // CVref = 0.500*Vdd
    VRCONbits.VREN = 1;         // enable voltage reference
                                // -> CVref = 2.5 V (if Vdd = 5.0 V)

    /*** Main loop ***/
    for (;;)
    {
        // test inputs 1 (C1IN+) and 2 (C2IN+)
        CMCON0bits.CIS = 1;      // select C1IN+ and C2IN+ pins as -refs
        __delay_us(10);          // wait 10us to settle
        sLED1 = CMCON0bits.C1OUT; // turn on LED1 if C1IN+ > CVref
        sLED2 = CMCON0bits.C2OUT; // turn on LED2 if C2IN+ > CVref

        // test inputs 3 (C2IN-) and 4 (C1IN-)
        CMCON0bits.CIS = 0;      // select C1IN- and C2IN- pins as -refs
        __delay_us(10);          // wait 10us to settle
        sLED3 = CMCON0bits.C2OUT; // turn on LED2 if C2IN- > CVref
        sLED4 = CMCON0bits.C1OUT; // turn on LED3 if C1IN- > CVref

        // display test results
        PORTA = sPORTA.RA;        // copy shadow registers to ports
        PORTC = sPORTC.RC;
    }
}

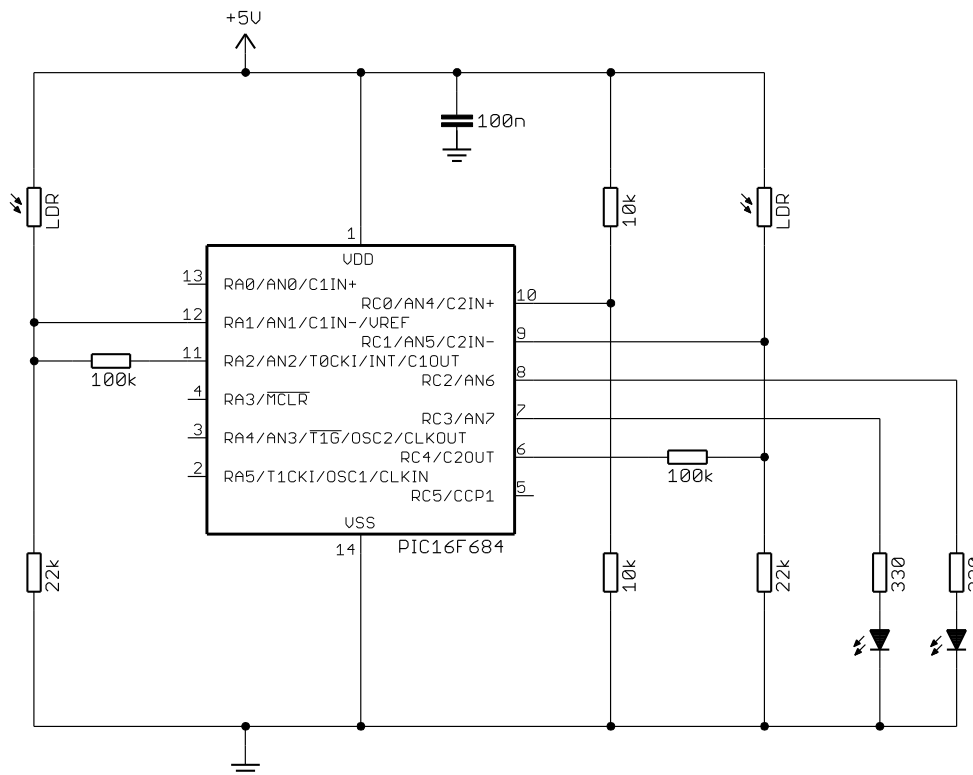
```

Using Interrupts with Two Comparators

As described in detail in [mid-range lesson 11](#), each of the 16F684's comparators can be selected independently as an interrupt source.

Briefly, there are now two interrupt enable bits, C1IE and C2IE, and two interrupt flags, C1IF and C2IF. Otherwise, the comparator interrupts are used in the same way as on the 12F629.

To illustrate this, we'll use the circuit shown below, which once again incorporates hysteresis, to minimise the number of interrupts triggered as the comparator inputs cross the threshold level. This is done by feeding the comparator's outputs, available on the C1OUT and C2OUT pins, back into the inputs (C1IN- and C2IN-) via 100 kΩ resistors.



To build it with the [Gooligum training board](#), connect the supplied photocell, 10 kΩ, 22 kΩ and 100 kΩ resistors as shown, using the solderless bread-board and the following pins on the 16-pin header:

C1IN- = pin 9 ('GP/RA/RB1'),

C1OUT = pin 13 ('GP/RA/RB2'),

C2IN+ = pin 10 ('RC0'),

C2IN- = pin 12 ('RC1'),

C2OUT = pin 6 ('RC4'),

VDD = pin 15 ('+V') and ground = pin 16 ('GND').

You also need to place a shunt in position 3 ('C1IN-') of JP25, to connect photocell PH2 (on the right of the board) to C1IN-.

Finally, close jumpers JP18 and JP19 to connect LEDs to RC2 and RC3.

The external outputs are only available in comparator mode 6 (110), and since the internal voltage reference is not available as in input in that mode, we have to use a external voltage divider, formed by the 10 kΩ resistors connected to C2IN+, which creates a 2.5 V reference.

As we did in the single-comparator interrupt example, we'll use the interrupt service routine to turn on an LED when its associated comparator input is higher than the reference level (corresponding to bright light).

XC8 implementation

As usual, we need to setup the I/O port (PORTC, in this case), so that the comparator input pins on PORTC (C2IN+ and C2IN-) are configured as inputs, and the other pins as outputs:

```
// configure ports
PORTC = 0;
sPORTC.RC = 0;
TRISC = 0b000011;

// start with LEDs off
// (update shadow)
// configure RC0/C2IN+ and RC1/C2IN- as inputs,
// rest of PORTC as outputs
```

There is no need to explicitly configure the comparator input pins on PORTA (C1IN- is used in this example) as inputs, as all pins are inputs by default.

To select comparator mode 6, with the outputs inverted (to create the positive feedback necessary for hysteresis), we have:

```
// configure comparators
CMCON0bits.CM = 0b110;           // select mode 6:
                                   // C1 -ref is C1IN-,
                                   // C1 +ref is C2IN+
                                   // C2 -ref is C2IN-,
                                   // C2 +ref is C2IN+,
                                   // both external outputs enabled,
                                   // both comparators on
CMCON0bits.C1INV = 1;           // C1 output inverted
                                   // -> C1OUT = 1 if C1IN- > C2IN+
CMCON0bits.C2INV = 1;           // C2 output inverted
                                   // -> C2OUT = 1 if C2IN- > C2IN+
```

To enable the external comparator outputs, we must clear the corresponding TRIS bits:

```
// enable comparator external outputs:
TRISAbits.TRISA2 = 0;           // C1OUT (RA2)
TRISCbits.TRISC4 = 0;           // C2OUT (RC4)
```

The interrupts are enabled in the same way as before, except that we now have two comparators:

```
// enable interrupts
INTCONbits.PEIE = 1;           // enable peripheral interrupts
ei();                           // enable global interrupts
                                   // enable comparator interrupts:
CMCON0;                         // read CMCON0 to clear mismatch
PIR1bits.C1IF = 0;             // clear interrupt flags
PIR1bits.C2IF = 0;
PIE1bits.C1IE = 1;             // set enable bits
PIE1bits.C2IE = 1;
```

The interrupt service routine is much the same as before, but with one important difference – we now have to respond to two possible input sources, so we have to check each comparator's interrupt flag, to ensure that we're servicing the correct interrupt source:

```
/****** INTERRUPT SERVICE ROUTINE *****/
void interrupt isr(void)
{
    // Service all triggered interrupt sources

    if (PIR1bits.C1IF)
    {
        //*** Service Comparator 1 interrupt
        //
        // Triggered on any comparator 1 output change,
        // caused by C1IN- input crossing C2IN+ threshold
        //
        CMCON0;                   // read CMCON0 to clear mismatch condition
        PIR1bits.C1IF = 0;        // clear interrupt flag

        // turn on LED 1 if C1IN- < C2IN+
        sLED1 = ~CMCON0bits.C1OUT; // display inverse of comparator 1 output
    }
}
```

```

if (PIR1bits.C2IF)
{
    /*** Service Comparator 2 interrupt
    /**
    /**   Triggered on any comparator 2 output change,
    /**   caused by C2IN- input crossing C2IN+ threshold
    /**
    CMCON0;                // read CMCON0 to clear mismatch condition
    PIR1bits.C2IF = 0;      // clear interrupt flag

    // turn on LED 2 if C2IN- < C2IN+
    sLED2 = ~CMCON0bits.C2OUT; // display inverse of comparator 2 output
}
}

```

The main loop still does nothing but continually copy the contents of a shadow register (which is updated by the interrupt service routine) to the output pins, but this time we're using PORTC instead of GPIO:

```

/*** Main loop ***/
for (;;)
{
    // continually copy shadow PORTC to port
    PORTC = sPORTC.RC;
}

```

Given that the program is otherwise very similar to the single-comparator interrupt example presented earlier, there is no need to present the full listing here. All of the source code is of course bundled with these tutorials.

Wake-up on Dual Comparator Change

Given that each of the 16F684's two comparators can be used as an interrupt source, it follows that each comparator can be used to wake the PIC from sleep mode.

This can be useful when you wish to monitor two inputs, and wake the PIC when either crosses a threshold.

Or, you could use the two comparators to monitor a single input, comparing it against two thresholds (one for each comparator), waking the PIC when either threshold is crossed.

These thresholds could be the upper and lower limits of an allowed range; the PIC waking and indicating an "alarm" condition, whenever the input goes outside these limits.

The circuit on the right was used in [mid-range lesson 11](#) to demonstrate wake-up when an input crosses either of two limits.

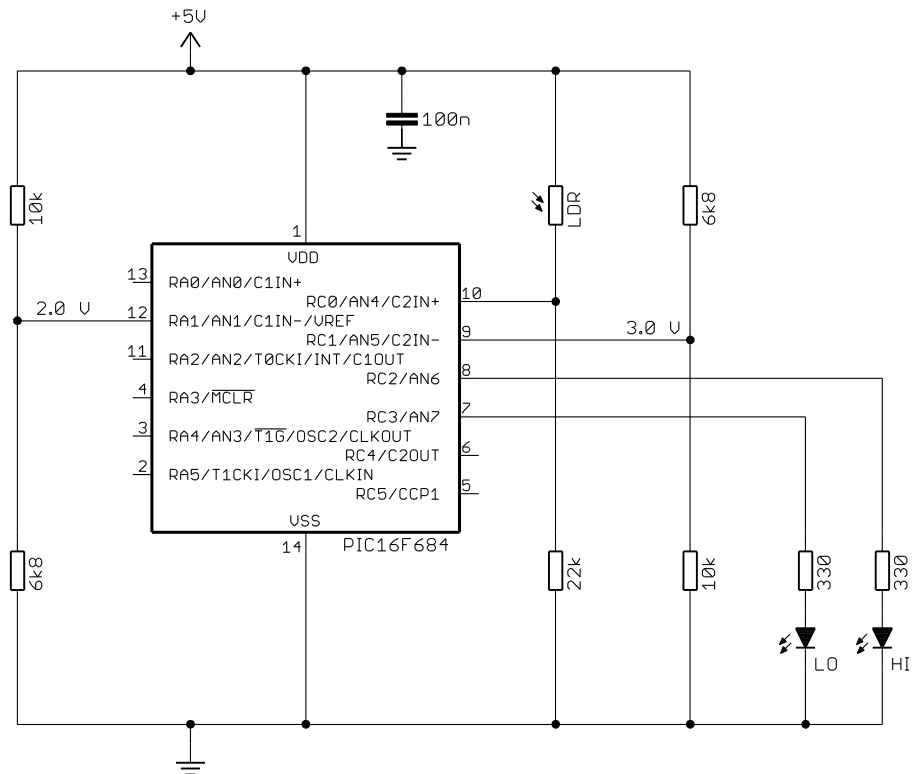
You can build it with the [Gooligum training board](#), by connecting the supplied 6.8 k Ω and 10 k Ω resistors as shown, using the solderless breadboard and the following pins on the 16-pin header:

C1IN- = pin 9 ('GP/RA/RB1'),

C2IN- = pin 12 ('RC1'),

VDD = pin 15 ('+V') and ground = pin 16 ('GND').

Move the shunt in JP25 to position 2 ('C2IN+'), to connect photocell PH2 to C2IN+, and leave JP18 and JP19 closed, connecting the LEDs to RC2 and RC3.



The most appropriate comparator mode for this application is 3 (011), where the common input is on C2IN+ and the two thresholds are defined by the voltages on C1IN- and C2IN-. In this example, a voltage divider is used to generate the low threshold of 2.0 V on C1IN-, and another voltage divider generates the high threshold of 3.0 V on C2IN-.

The LED on RC3 is used to indicate the “not enough light” condition and the LED on RC2 indicates “too much light”.

When either of the thresholds is crossed (in either direction), the PIC should wake, and if the input is outside the allowed range, the appropriate LED should be turned on, until the input is back within the allowed range. The PIC can then go back into sleep mode, to wait for the next threshold crossing.

XC8 implementation

The comparator mode is selected as usual, with the inversion bits configured so that each output is a ‘1’ only when an error condition (input too high or low) exists:

```
// configure comparators
CMCON0bits.CM = 0b011;           // select mode 3:
                                   // C1 -ref is C1IN-,
                                   // C1 +ref is C2IN+,
                                   // C2 -ref is C2IN-,
                                   // C2 +ref is C2IN+,
                                   // no external outputs,
                                   // both comparators on

CMCON0bits.C1INV = 1;             // C1 output inverted
                                   // -> C1OUT = 1 if C2IN+ < C1IN-
CMCON0bits.C2INV = 0;             // C2 output not inverted
                                   // -> C2OUT = 1 if C2IN+ > C2IN-
```

Wake-up on comparator change is enabled in the same way as in the single-comparator example, except that there are now two comparators:

```
// enable comparator interrupts (for wake on change)
INTCONbits.PEIE = 1;           // enable peripheral interrupts
PIE1bits.C1IE = 1;            // and comparator interrupts
PIE1bits.C2IE = 1;
```

In the main loop, we need to test each comparator output, and light the corresponding LED as long as that comparator output remains high.

The C code logically equivalent to the assembler code presented in [mid-range lesson 11](#) is:

```
// test for and wait while input low
while (CMCON0bits.C1OUT)      // while C2IN+ < C1IN-
    LO = 1;                   // turn on "low" LED

// test for and wait while input high
while (CMCON0bits.C2OUT)      // while C2IN+ > C2IN-
    HI = 1;                   // turn on "high" LED
```

Very short and simple!

In fact, it is a little too short; the assembler version included ‘banksel’ directives which are not strictly necessary, because the registers involved in this code fragment (CMCON0 and PORTC) are in the same bank. The XC8 compiler does not generate unnecessary bank selection code. This means that the generated C code is shorter, and faster, than the hand-written assembler version – in this case, a little too fast!

It turns out that the sudden additional current drain caused by turning on either LED connected to PORTC creates enough disruption to momentarily affect the comparator inputs. To generate consistent, correct results when reading the comparator outputs, it is necessary to add a short delay between lighting the LED and reading the comparator input.

The following code works correctly:

```
// test for and wait while input low
while (CMCON0bits.C1OUT)      // while C2IN+ < C1IN-
{
    LO = 1;                   // turn on "low" LED
    __delay_us(5);           // short delay to settle
}

// test for and wait while input high
while (CMCON0bits.C2OUT)      // while C2IN+ > C2IN-
{
    HI = 1;                   // turn on "high" LED
    __delay_us(5);           // short delay to settle
}
```

When these loops have completed, we know that the input must be no longer too high or too low⁶, so we can turn off any LEDs that had been lit:

```
// turn off LEDs
PORTC = 0;
```

⁶ unless it is changing very quickly...

Then we can enter sleep mode, after clearing any existing comparator mismatch condition (by reading CMCON0) and clearing the comparator interrupt flags:

```
// go into standby (low power) mode
CMCON0;           // read CMCON0 to clear comparator mismatch
PIR1bits.C1IF = 0; // clear comparator interrupt flags
PIR1bits.C2IF = 0;
SLEEP();          // enter sleep mode
```

And when the device wakes from sleep mode, we can start the main loop again, to detect which threshold was crossed.

Complete program

Here is how these fragments fit together:

```
/******
 *
 *   Description:      Lesson 6, example 10
 *
 *   Demonstrates wake-up on dual comparator change
 *
 *   Device wakes when input crosses a low or high threshold
 *
 *   Turns on Low  LED when C2IN+ < C1IN-
 *               or High LED when C2IN+ > C2IN-
 *   then sleeps until the next change
 *
 *   (thresholds are set by external voltage dividers)
 *
 *****/
 *
 *   Pin assignments:
 *       C2IN+ = voltage to be measured (e.g. pot output or LDR)
 *       C1IN- = low threshold (resistor divider)
 *       C2IN- = high threshold (resistor divider)
 *       RC3   = "Low" LED
 *       RC2   = "High" LED
 *
 *****/

#include <xc.h>

#define _XTAL_FREQ 4000000 // oscillator frequency for _delay()

/***** CONFIGURATION *****/
// ext reset, no code or data protect, no brownout detect,
// no watchdog, power-up timer enabled, int clock with I/O,
// no failsafe clock monitor, two-speed start-up disabled
__CONFIG(MCLR_ON & CP_OFF & CPD_OFF & BOREN_OFF &
        WDTE_OFF & PWRTE_ON & FOSC_INTOSCIO &
        FCMEN_OFF & IESO_OFF);

// Pin assignments
#define LO      PORTCbits.RC3 // "Low" LED
#define nLO     3             // (port bit 3)
#define HI      PORTCbits.RC2 // "High" LED
#define nHI     2             // (port bit 2)
```

```

/***** MAIN PROGRAM *****/
void main()
{
    /*** Initialisation ***/

    // configure ports
    PORTC = 0;                // start with both LEDs off
    TRISC = ~(1<<nLO|1<<nHI); // configure LED pins (only) as outputs

    // configure comparators
    CMCON0bits.CM = 0b011;    // select mode 3:
                                //   C1 -ref is C1IN-,
                                //   C1 +ref is C2IN+,
                                //   C2 -ref is C2IN-,
                                //   C2 +ref is C2IN+,
                                //   no external outputs,
                                //   both comparators on
    CMCON0bits.C1INV = 1;     // C1 output inverted
                                //   -> C1OUT = 1 if C2IN+ < C1IN-
    CMCON0bits.C2INV = 0;     // C2 output not inverted
                                //   -> C2OUT = 1 if C2IN+ > C2IN-

    // enable comparator interrupts (for wake on change)
    INTCONbits.PEIE = 1;      // enable peripheral interrupts
    PIR1bits.C1IE = 1;        // and comparator interrupts
    PIR1bits.C2IE = 1;

    /*** Main loop ***/
    for (;;)
    {
        // test for and wait while input low
        while (CMCON0bits.C1OUT) // while C2IN+ < C1IN-
        {
            LO = 1;             // turn on "low" LED
            __delay_us(5);      // short delay to settle
        }

        // test for and wait while input high
        while (CMCON0bits.C2OUT) // while C2IN+ > C2IN-
        {
            HI = 1;             // turn on "high" LED
            __delay_us(5);      // short delay to settle
        }

        // turn off LEDs
        PORTC = 0;

        // go into standby (low power) mode
        CMCON0;                 // read CMCON0 to clear comparator mismatch
        PIR1bits.C1IF = 0;      // clear comparator interrupt flags
        PIR1bits.C2IF = 0;
        SLEEP();                // enter sleep mode
    }
}

```

Summary

This has been a very long lesson, for such an apparently simple peripheral.

We have seen that it is possible to effectively utilise the comparators and voltage reference available on older mid-range PICs, such as the 12F629 and 16F684, using the XC8 compiler – we were able to use the comparators in a number of modes, with and without the programmable voltage reference, and we saw that it was straightforward to use C to implement comparator interrupts and to configure the comparator to wake the device from sleep.

But as the last example showed, there can be a downside, in that you may not be aware of exactly what the code generated by your C compiler is doing – we had to add short delays into the polling loops, because the C compilers optimised the code a little “too well”! This isn’t a bug; the compilers are doing what they should, but it shows that sometimes what’s happening in detail, at a low level normally hidden by the C compiler, can be important. In assembler, you have no choice but to be across all that low-level detail...

The [next lesson](#) will focus on driving 7-segment displays (revisiting the material from [mid-range lesson 12](#)), showing how lookup tables and multiplexing can be implemented using XC8.